Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

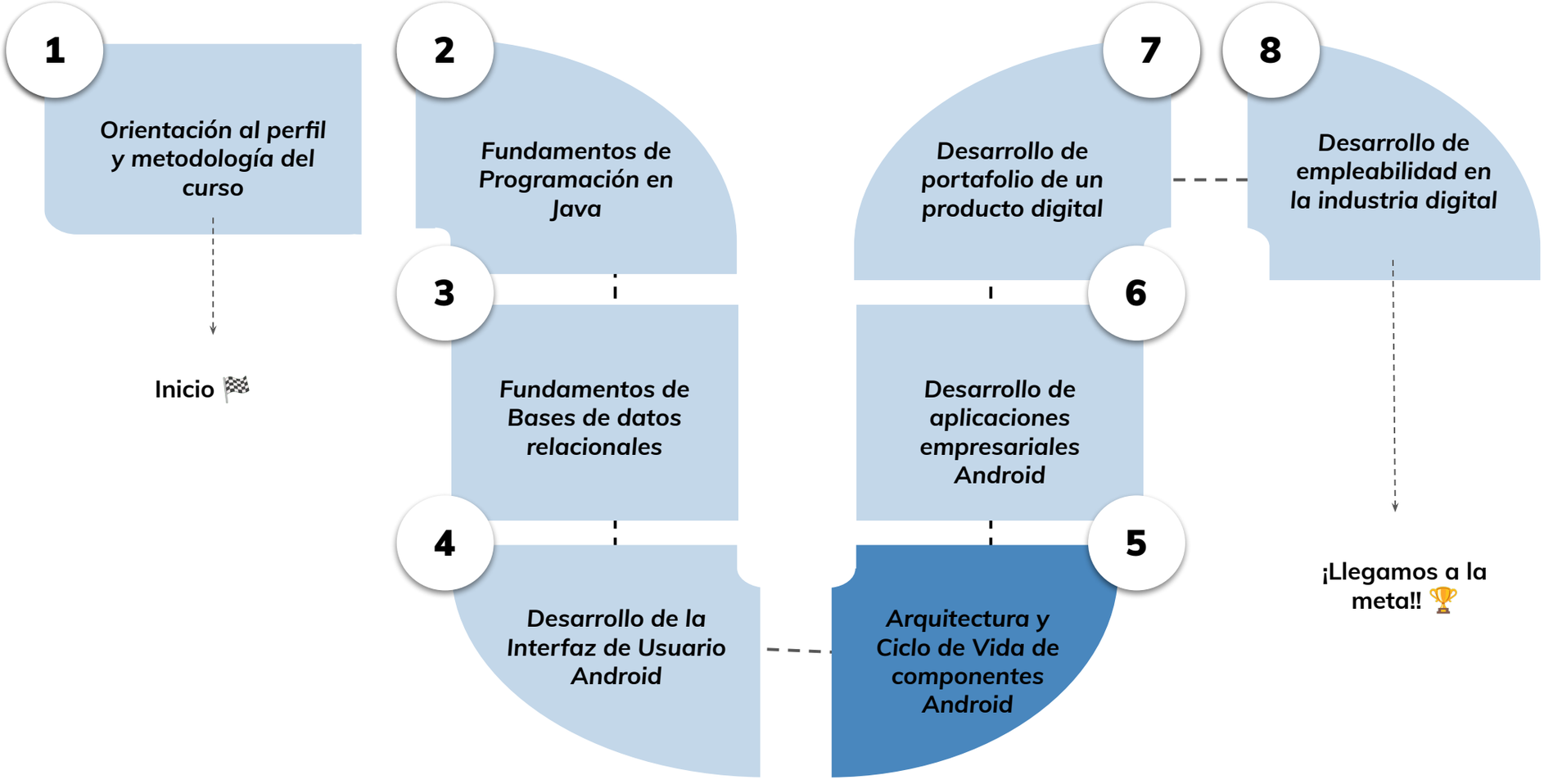
Programación asíncrona en android -Parte I

AE: Utilizar elementos de la programación asíncrona acorde al lenguaje kotlin para dar solución a un problema.

alkemy

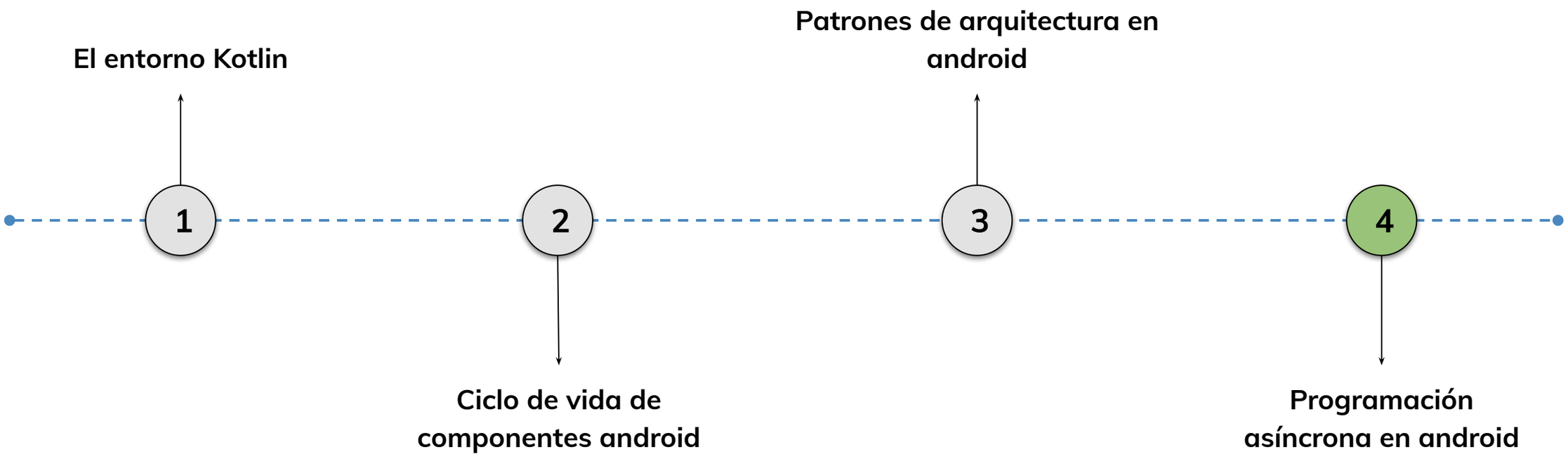
Hoja de ruta

*¿Cuáles **módulos** conforman el programa?*



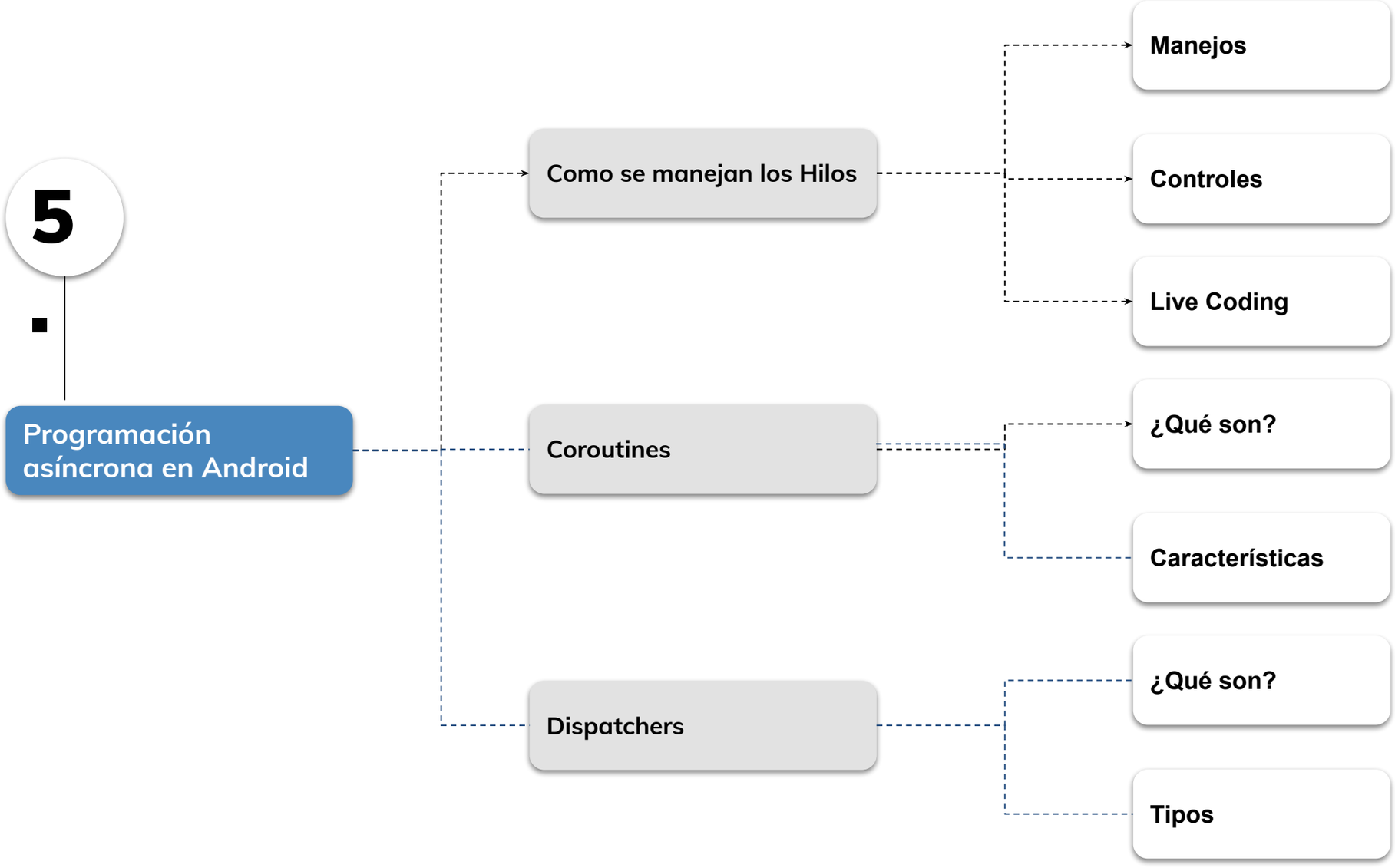
Roadmap de la lección

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- *Entender cómo se manejan los hilos en Android*
- *Comprender cómo se controlan los flujos de ejecución*
- *Ver cómo usamos esto hilos en el desarrollo de aplicaciones*
- *Entender las Corrutinas y sus usos*
- *Comprender los tipos de dispatchers*
- *Saber los diferentes usos de las mismas*

Rompehielo🥶

Contexto: 🙌

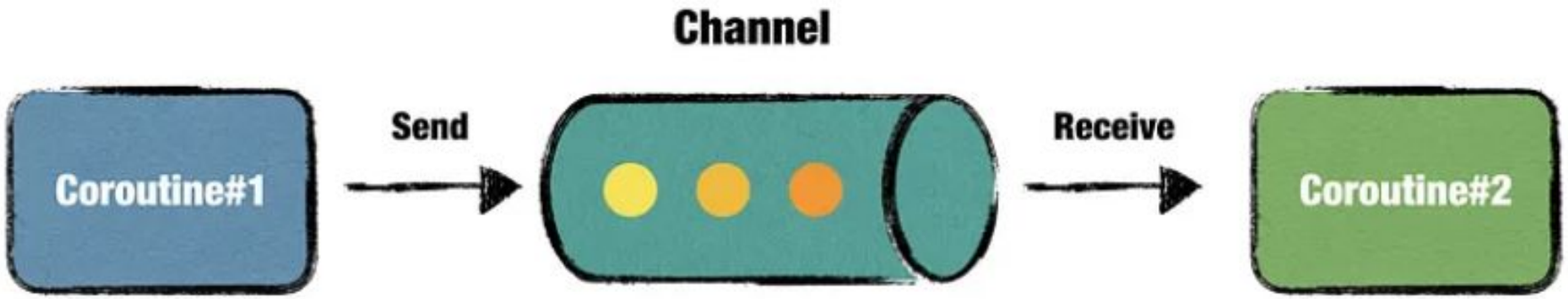
Existen varias formas de controlar hilos de ejecución en Android pero en muchas ocasiones vamos a necesitar enviar información como resultados o actualizar alguna vista.

Consigna: ✍️

¿De qué formas podemos saber si un hilo de ejecución ya está listo para devolver la información que necesitamos?

› Cómo se manejan los Hilos

HILOS



alkemy

Manejos de Hilos

¿Cómo?:

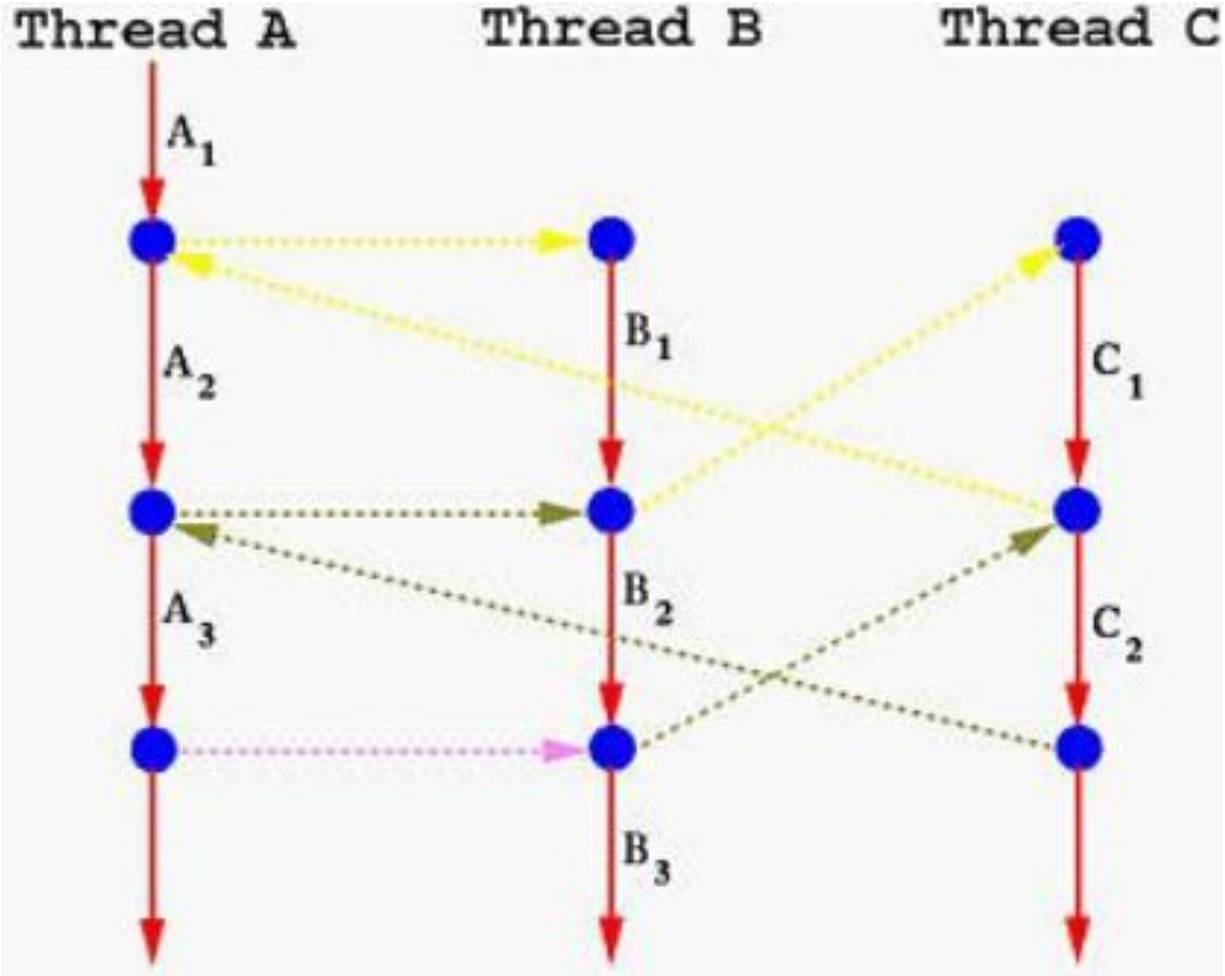
En Android existen múltiples librerías y funcionalidades que permiten lanzar y controlar tareas en segundo plano, algunas están en desuso y tras aparecieron para dar mejor soporte a funciones modernas.
Actualmente el estándar suele ser usar coroutinas y aplicar el modelo de observador pero esto puede ser complejo y no ideal para todas la tareas que queramos correr en segundo plano así que vamos a explorar otras soluciones

HILOS EN COROUTINES

¿Cómo?:

En Android existen múltiples librerías y funcionalidades que permiten lanzar y controlar tareas en segundo plano, algunas están en desuso y tras aparecieron para dar mejor soporte a funciones modernas.
Actualmente el estándar suele ser usar coroutinas y aplicar el modelo de observador pero esto puede ser complejo y no ideal para todas la tareas que queramos correr en segundo plano así que vamos a explorar otras soluciones

HILOS EN COROUTINES



HILOS EN COROUTINES

Control de hilo Principal:
El hilo principal es el hilo de ejecución en el que se realiza la interfaz de usuario. Es esencial que las operaciones que afecten la interfaz de usuario se realicen en el hilo principal. Puedes usar la función runOnUiThread para ejecutar código en el hilo principal

```
runOnUiThread {
    // Código para actualizar la interfaz de usuario
}
```

HILOS EN COROUTINES

Handler:
Un Handler permite programar tareas para su ejecución en el hilo principal. Puedes crear un Handler vinculado al hilo principal y utilizarlo para enviar mensajes y tareas:

```
val handler = Handler(Looper.getMainLooper())
handler.post {
    // Código para ejecutar en el hilo principal
}
```

HILOS EN COROUTINES

AsyncTask:
Aunque AsyncTask está en desuso en versiones más recientes de Android, sigue siendo útil para tareas en segundo plano que interactúan con la interfaz de usuario. Puedes crear una clase que extienda AsyncTask y anular los métodos doInBackground y onPostExecute:

```
class MyAsyncTask : AsyncTask<Void, Void, ResultType>() {
    override fun doInBackground(vararg params: Void): ResultType {
        // Realiza tareas en segundo plano
    }

    override fun onPostExecute(result: ResultType) {
        // Actualiza la interfaz de usuario con el resultado
    }
}
```

HILOS EN COROUTINES

Kotlin Coroutines:
Las coroutines son una forma moderna y eficiente de manejar hilos en Android. Con el uso de la biblioteca kotlinx.coroutines, puedes realizar tareas en segundo plano de manera más concisa y segura, y volver al hilo principal cuando sea necesario. Por ejemplo:

```kotlin
// Ejecutar una coroutine en segundo plano
GlobalScope.launch(Dispatchers.IO) {
    // Tareas en segundo plano
    val result = performBackgroundTask()

    // Actualizar la interfaz de usuario en el hilo principal
    withContext(Dispatchers.Main) {
        updateUI(result)
    }
}
```

HILOS EN COROUTINES

Executor:
Los Executors simplifican la gestión de hilos y son útiles para tareas que deben ejecutarse en segundo plano, como operaciones de red, acceso a bases de datos u otras tareas que no deben bloquear el hilo principal de la interfaz de usuario.
Existen diferentes implementaciones de Executor, pero uno de los más utilizados en Android es ThreadPoolExecutor. A continuación, te mostraré un ejemplo de cómo crear y usar un Executor en Kotlin para Android

```kotlin
import java.util.concurrent.Executor
import java.util.concurrent.Executors

class MyTask : Runnable {
    override fun run() {
        // Realiza la tarea en segundo plano
    }
}

// Crear un Executor utilizando Executors
val executor: Executor = Executors.newFixedThreadPool(2) // Ejecuta
hasta 2 hilos simultáneos

// Programar la ejecución de una tarea en segundo plano
executor.execute(MyTask())
```

LIVE CODING

Ejemplo en vivo

Correr una llamada HTTP en el main thread:

Vamos a correr una simple llamada en el hilo principal para ver qué ocurre

1. *Crear un cliente con OkHttpClient()*
2. *Crear un request con un Url*
3. *Hacer una llamada con el cliente*
4. *Correr la llamada en el MainThread*
5. *Ver que pasa cuando corremos el código*
6. *¿Cómo podemos arreglarlo?*

Tiempo: 40 min

¿Alguna consulta?

alkemy

Momento:

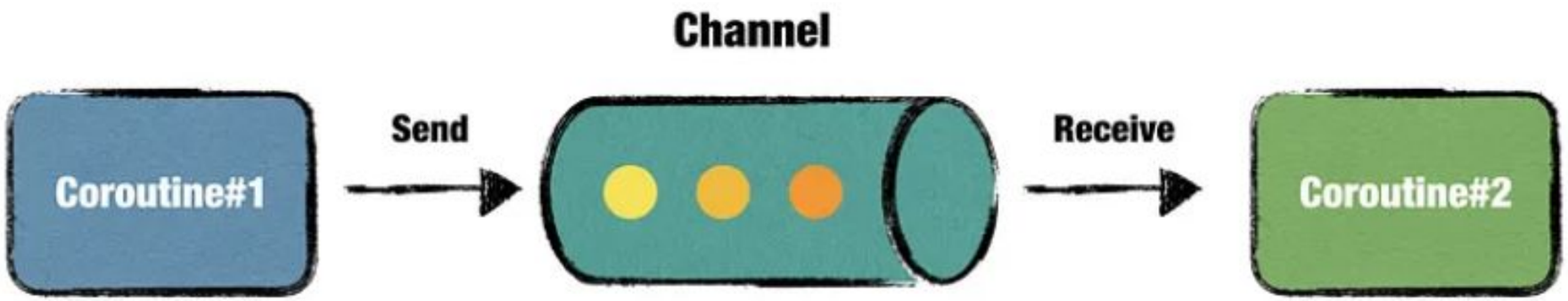
Time-out!

⏰5 -10 min.

alkemy

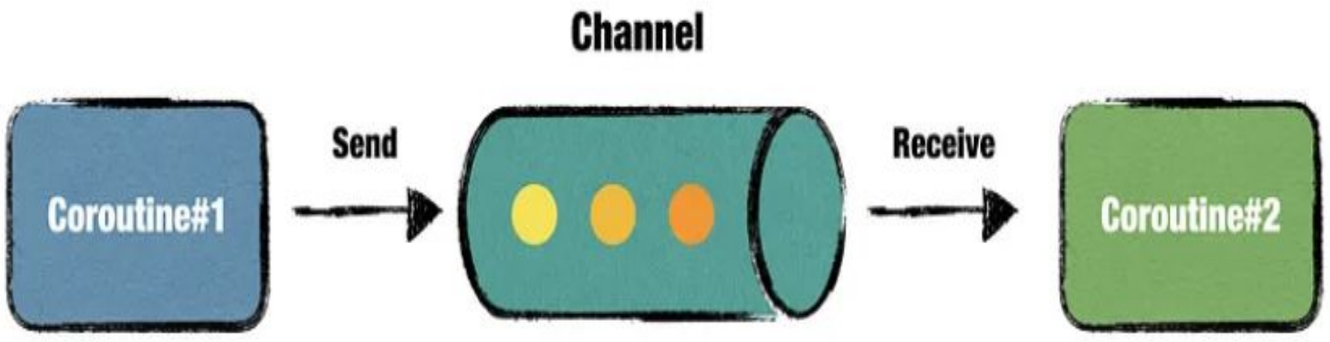
› Corrutinas

COROUTINES



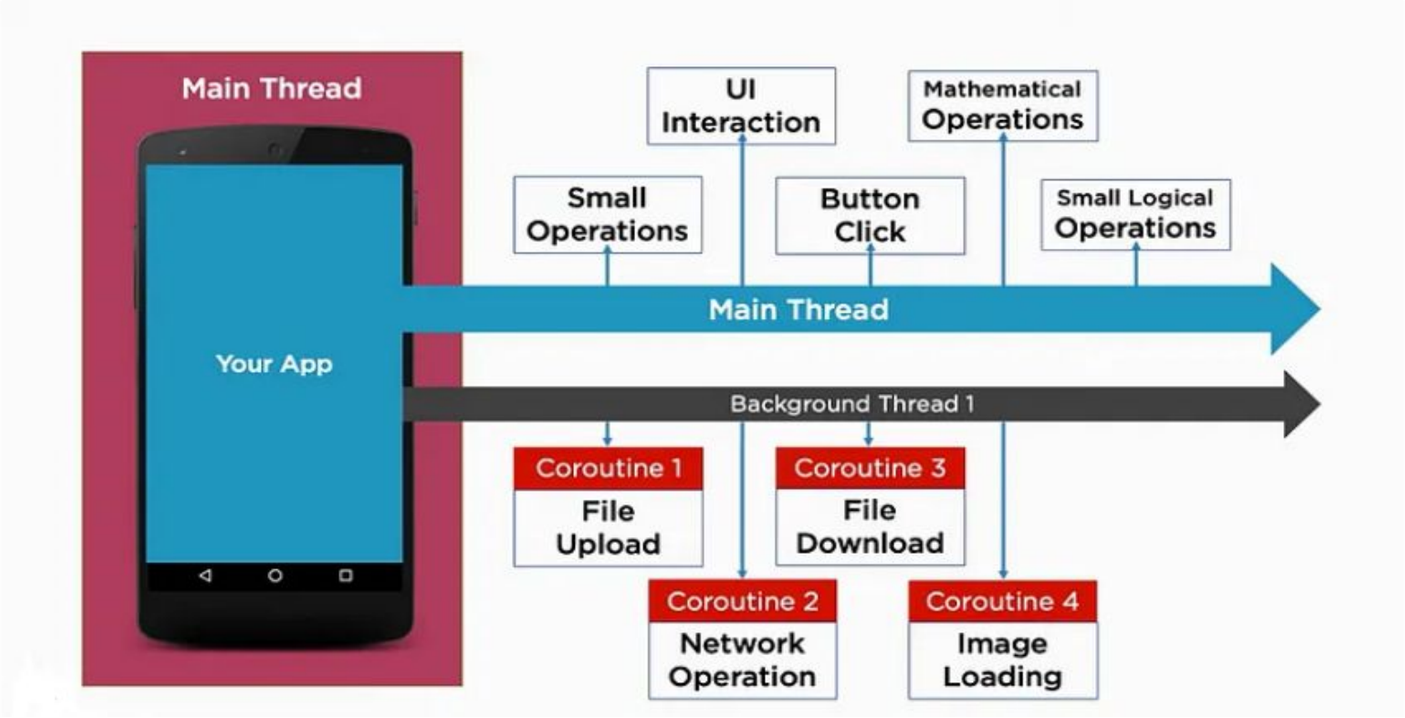
¿Qué son las Coroutines?

¿Qué son las coroutines?
Según la propia [documentación oficial](), una coroutine es en sí misma un hilo liviano. Es decir, se vale de hilos o *pool* de hilos para ejecutar código de manera concurrente y de forma muy eficiente optimizando el uso de recursos.



¿Qué son las Coroutines?

Thread vs. Coroutines: Choosing the Right Concurrency Approach in Kotlin Android



¿Qué ventajas ofrecen las Coroutines?

Desde el punto de vista del desarrollador: Las coroutines permiten implementar código asincrónico con un estilo idéntico al código sincrónico. Gracias a que su mecanismo interno hace uso de la interfaz *Continuation* cada vez que se suspende un hilo, su implementación se vuelve más legible al no ser nosotros los que tenemos que lidiar con el uso de *callbacks* o implementaciones de interfaces con clases anónimas.

Desde el punto de vista del rendimiento : Las coroutines permiten la ejecución de miles y hasta millones de subprocesos concurrentemente con un uso de recursos eficiente haciendo más robusta la aplicación al ser más difícil de alcanzar un error que indique falta de memoria.

Desde el punto de vista de la implementación: las coroutines permiten una manipulación sencilla de los hilos de ejecución valiéndose de los *Dispatchers*, los cuales nos permiten determinar explícitamente el hilo que deseamos que ejecute un bloque específico de código.

Características:

Sintaxis Sencilla:

Utilizan palabras clave como launch y async para iniciar tareas en segundo plano, lo que facilita la lectura y escritura de código asincrónico.
También tienen una más clara forma de declarar los hilos de ejecución y cuando se van a ejecutar

Características:

Más estable :

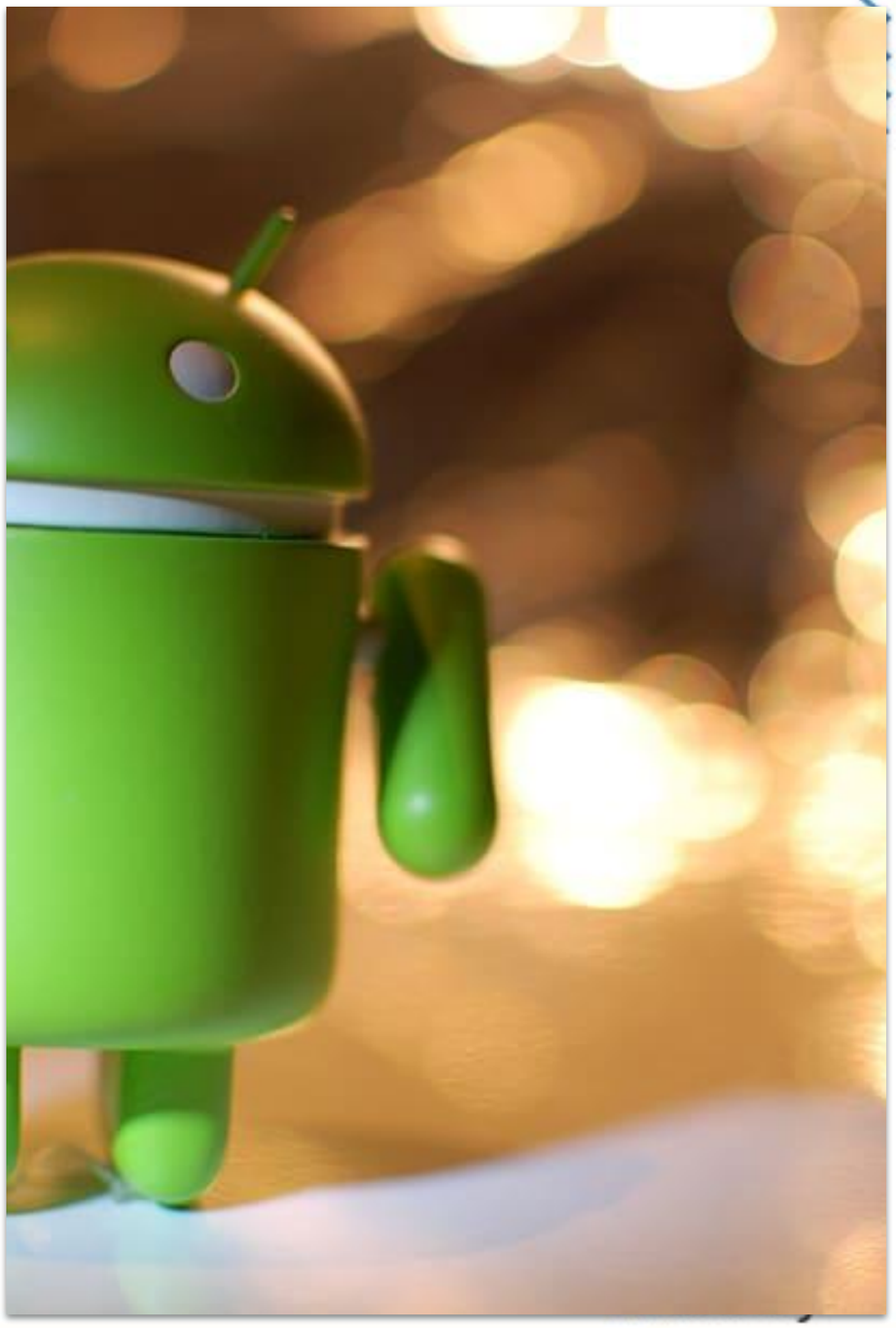
Manejo de Excepciones Mejorado: Las coroutines ofrecen un manejo de excepciones más sencillo y robusto.

Cancelación de Tareas: Puedes cancelar coroutines de manera segura cuando ya no son necesarias, evitando fugas de memoria.

Características:

Funciones de Suspensión:

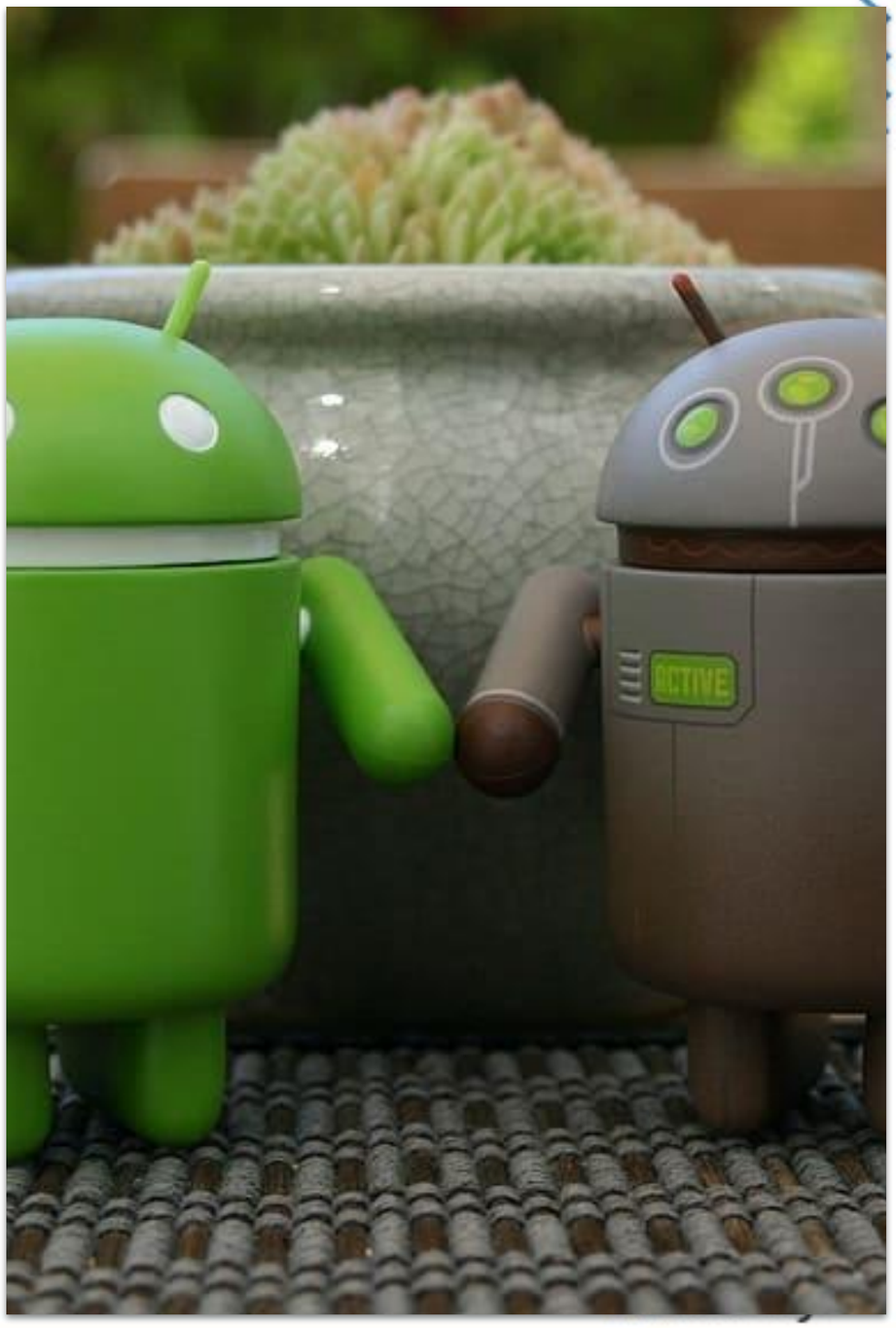
Puedes hacer que una función espere asincrónicamente utilizando la palabra clave suspend, lo que permite pausar la ejecución hasta que se complete una tarea.
Esto es vital para que podamos llamar funciones en un thread secundario.



Características:

Manejo de Hilos Simplificado:

Los "dispatchers" permiten especificar en qué tipo hilo se ejecutará una coroutine, lo que simplifica el manejo de hilos y evita bloquear el hilo principal.
Existen varios tipos de Dispatchers que se llaman dependiente del tipo de tarea que se va a correr.



Dispatchers

Main:

Este dispatcher se utiliza para realizar operaciones en el hilo principal de la interfaz de usuario. Es fundamental para actualizar la interfaz de usuario con datos obtenidos en segundo plano
Lo podemos usar para lanzar tareas en el thread principal.

```
withContext(Dispatchers.Main) {
    // Actualizar la interfaz de usuario
}
```

alkemy

Dispatchers

IO:

Este dispatcher se utiliza para realizar operaciones de E/S (entrada/salida) en hilos en segundo plano. Es ideal para tareas que no requieren acceso directo a la interfaz de usuario pero principalmente para llamadas a APIs, bases de datos o cualquier dato externo

```
withContext(Dispatchers.IO) {
    // Realizar operaciones de E/S
}
```

alkemy

Dispatchers

Default:

Este dispatcher se utiliza para realizar tareas de CPU intensivas en hilos en segundo plano. Es adecuado para cálculos y procesamiento intensivo que no bloquearán el hilo principal.

```
withContext(Dispatchers.Default) {
    // Realizar cálculos intensivos de CPU
}
```

alkemy

Dispatchers

Unconfined:

Este dispatcher no impone ningún hilo específico y puede ser útil cuando necesitas que una coroutine continúe en el hilo en el que se llamó.

```
withContext(Dispatchers.Unconfined) {
    // Continuar en el hilo actual
}
```

alkemy

Dispatchers

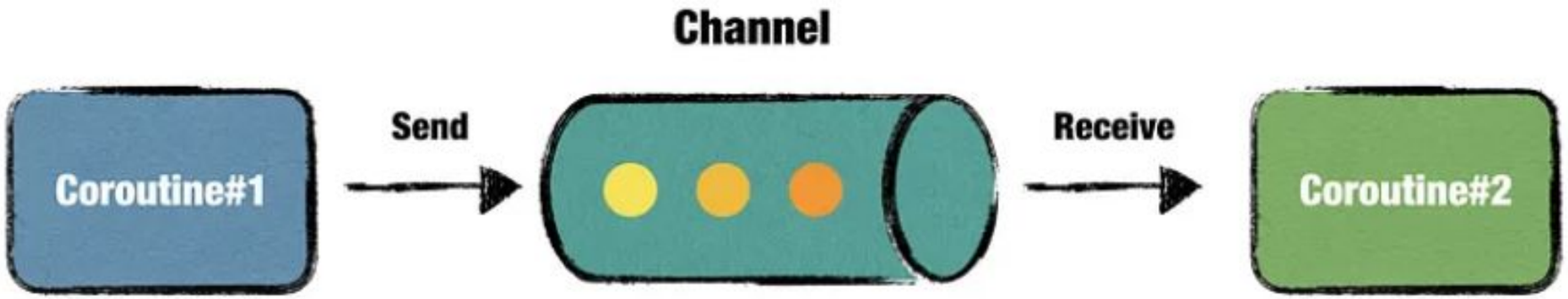
Main.immediate:

Similar a Dispatchers.Main, pero ejecuta la tarea de manera inmediata, lo que puede ser útil en pruebas unitarias.

```
withContext(Dispatchers.Main.immediate) {
    // Realizar tareas de prueba en el hilo principal
}
```

alkemy

EJEMPLOS



alkemy

EJEMPLO

Por ejemplo, simulemos un programa que marca las palabras encontradas por un usuario en una sopa de letras. Al mismo tiempo le temporizamos 10 segundos como plazo.

```
// Importar componentes de corrutinas
 import kotlinx.coroutines.*
 fun main() {
 // 1. Inicio
        println("¡Go!")
// 2. Buscar palabras en el background
 GlobalScope.launch {

          (1..5).forEach {
           delay(300) println("¡Palabra $it encontrada!")
             }
}

// 3. Iniciar temporizador en foreground
 for (i in 10 downTo 1)
              { println("${i}s") Thread.sleep(100)
        }

 // 4. Tiempo fuera
println("Se terminó el tiempo") }
```

La solución propuesta consiste en:

- **Iniciar la corrutina con GlobalScope.launch{}.**

  Esta cuenta las palabras encontradas. Usamos la función de suspención delay() para simular que el usuario se demora 300 milisegundos encontrando cada palabra

- Iniciar un bucle for en reversa para simular el temporizador de 10 a 1. Usamos el método Thread.sleep() de la librería estándar de Java, para dormir el hilo principal (main)

```
¡Go!
10s
9s
8s
7s
¡Palabra 1 encontrada!
6s
5s
4s
¡Palabra 2 encontrada!
3s
2s
1s
¡Palabra 3 encontrada!
Se terminó el tiempo
```

EJEMPLO

Iniciar Una Corrutina Con **runBlocking()**

runBlocking() inicia una nueva corrutina y bloquea su hilo contenedor, hasta que se ejecuten todas las sentencias de su bloque de código.

Por ejemplo, como faltaron 2 palabras en el ejemplo anterior, podemos usar runBlocking() al final para crear una corrutina que brinde unos 600 milisegundos más, mientras el usuario encuentra las palabras:

```
// 4. Tiempo fuera
("Se terminó el tiempo")
  runBlocking {
        delay(600)
 }
```

```
¡Go!
10s
9s
8s
7s
¡Palabra 1 encontrada!
6s
5s
4s
¡Palabra 2 encontrada!
3s
2s
1s
¡Palabra 3 encontrada!
Se terminó el tiempo
¡Palabra 4 encontrada!
¡Palabra 5 encontrada!
```

FUNCIONES SUSPENDIDAS

Las corrutinas se fundamentan en el uso de funciones de suspensión. Estas son funciones capaces de bloquear la ejecución de la corrutina mientras ejecutan su trabajo, y tienen la particularidad de retornar el resultado de la operación para su posterior uso. Esto permite que las operaciones asíncronas dentro de una corrutina sean programadas en código secuencial.

Es muy importante tener siempre presente que las funciones de suspensión solamente pueden ser ejecutadas dentro de una corrutina o dentro de otra función de suspensión. Este aspecto tiene mucho impacto en el desarrollo de corrutinas, ya que siempre va a tener que existir una estructura de corrutinas que se lancen desde el contexto adecuado y ejecuten las funciones necesarias que se definen en otros elementos del sistema.
Como se muestra a continuación, una función de suspensión maneja la misma estructura que cualquier función de Kotlin, haciendo uso de la palabra reservada suspend:

```
suspend fun
suspendingFunction() : Int
{
 // Long running task
return 0
}
```

View

Model

alkemy

LOS DISTINTOS SCOPES

- **GlobalScope**: es el *scope* más general. Permite ejecutar corrutinas mientras la aplicación también siga siendo ejecutada. Por este motivo, no deben estar atadas a ningún componente que pueda ser destruido.
- **View.lifecycleScope:** es un *scope* que existe a nivel de los componentes de las vistas: es decir, Actividades y Fragmentos. Este scope permite ejecutar corrutinas vinculadas al ciclo de vida del componente. Cada clase que extienda de Activity o Fragment tendrá por defecto el atributo lifecycleScope.
- **ViewModel.viewmodelScope:** es un scope que existe como propiedad por defecto de las clases que extiendan de ViewModel. Un ViewModel suele invocar operaciones asíncronas relacionadas a la información de una vista y, por este motivo, requieren de un *scope* distinto al de la vista misma.
- **Personalizadas, creadas con el constructor CoroutineScope() o MainScope**(): son *scopes* estándar que se instancian directamente en el código como variables. La diferencia entre los dos constructores radica en el *dispatcher* del *scope* que se crea. Es necesario cancelar explícitamente estos *scopes* cuando no se necesiten más.

View

Model

LIVE CODING

Ejemplo en vivo

Desarrollo de una Aplicación con Listado de Personas Utilizando Programación Asíncrona

1. **Requisitos Funcionales:**
   - La aplicación debe mostrar un listado de personas, donde cada persona tiene los siguientes datos: **nombre**, **apellido** y **edad**.
   - La aplicación debe implementar una **clase repositorio** que contenga los datos de las personas.
   - Se debe diseñar una **interfaz** que exponga dicho listado de personas.
2. **Interfaz de Usuario:**
   - La vista debe contener los siguientes elementos:
     - **2 TextView**: Uno para mostrar el nombre y apellido de cada persona, y otro para mostrar la edad.
     - **1 ProgressBar**: Para indicar que los datos se están cargando.
     - **1 Button**: Al presionar este botón, se iniciará la carga de los datos del listado de personas.

LIVE CODING

Ejemplo en vivo

Cargar Datos de Forma Asíncrona:

- Cuando el usuario presiona el **Button**, se debe iniciar el proceso de carga de los datos desde un repositorio. Este proceso debe ejecutarse de forma asíncrona utilizando **Coroutines** en Kotlin, con el fin de evitar que la interfaz de usuario se congele mientras se cargan los datos.
- La carga de los datos se debe realizar en un **hilo secundario** (coroutine) para no bloquear el hilo principal.
- Durante el proceso de carga, el **ProgressBar** debe estar visible para indicar que los datos se están cargando.
- Una vez que los datos estén cargados, el **ProgressBar** debe desaparecer y el listado de personas debe mostrarse en la interfaz de usuario.

.

LIVE CODING

Ejemplo en vivo

- **Detalles Técnicos:**
  a. **Repositorio**: Crear una clase Repositorio que contenga los datos de las personas (nombre, apellido y edad).
  b. **Coroutines**: Utilizar suspend y launch para ejecutar la carga de datos en un hilo secundario. Utilizar withContext(Dispatchers.Main) para actualizar la interfaz de usuario cuando los datos se hayan cargado.
  c. **Activity Principal**: En la actividad principal, gestionar la interacción del botón para iniciar la carga y actualizar los elementos de la interfaz de usuario (TextView y ProgressBar) de acuerdo al estado de la carga.

Este enfoque garantiza una experiencia fluida para el usuario al no bloquear la interfaz de usuario durante la carga de datos, utilizando programación asíncrona para optimizar el rendimiento.

Timpo 1 hora

LIVE CODING

Ejemplo en vivo

alkemy

LIVE CODING SOLUCIÓN

Ejemplo en vivo

GitHub

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ *Uso de hilos*
- ✓ *Control de hilos de ejecución*
- ✓ *Manejo de Threads*
- ✓ *Corrutinas*
- ✓ *Dispatchers*
- ✓ *Tipos de Hilos*

< ¡Muchas gracias! >

alkemy